

FORMATION ACCELEREE
INITIATION À LA PROGRAMMATION ET CONCEPTION DES APPLICATIONS
INFORMATIQUES

Facilitateur : **Odilaste TSHIYIRE**, Ingénieur en génie logiciel

OBJECTIFS

Les apprenants de cette formation devront être capables de :

- Comprendre la programmation et s'en sortir peu importe le langage qu'ils feront face dans la pratique (programmation) ;
- Faire la programmation d'une manière réfléchie et professionnelle ;
- Réaliser des projets finis et consommables par les clients

INTRODUCTION

PLAN DU COURS

Chapitre I : Cahier de charge

Chapitre II : Algorithmes

Chapitre III : Codage des algorithmes et programmation dans un langage donné

Chapitre IV : Tests et validation de l'application

CHAP I. CAHIER DE CHARGE

Définition

Le cahier des charges est, dans un contrat informatique ou un projet informatique, le document dans lequel le client exprime ses besoins, c'est-à-dire les caractéristiques fonctionnelles et techniques attendues de la solution informatique.

Le cahier des charges contient généralement les spécifications techniques (description de l'environnement de production dans lequel sera implémentée la solution, langage dans lequel devra être développé le logiciel, etc.) et les spécifications fonctionnelles (description des fonctionnalités attendues par le client) de la solution informatique objet du contrat, ou encore la description des interfaces qui devront être réalisées pour permettre à la solution d'échanger avec d'autres logiciels ou matériels constituant le système d'information du client.

Rubriques principales du cahier de charge

1. Présentation de l'entreprise prestataire du service/Vendeur du produit logiciel

Dans cette partie vous vous identifiez, vous l'entreprise(particulier) qui cherche à rendre les services ou à vendre le produit. **Exemple : cfr Annexe Cahier de charge**

2. Contexte et définition du problème

Vous vous dites peut-être qu'un développeur n'est intéressé que par la partie technique. Détrompez-vous, nombreux sont ceux qui aiment voir la finalité de leur travail. De plus, donner du sens au développement facilite la compréhension de vos besoins et de vos contraintes. Le dialogue plus tard n'en sera que plus facile. Enfin, en définissant le problème - en démontrant

ainsi le bien-fondé de votre requête - vous avez plus de chances de voir atterrir votre cahier des charges sur le haut de la pile des dossiers à traiter dans le bureau d'à côté.

3. Objectif du projet

Suite à l'explicitation du contexte, exprimez **le niveau de de l'attente des clients et les résultats attendus**. Les objectifs seront quantifiés.

4. Périmètre

Fixez ici les limites que vous donnez à votre projet. Etre capable d'identifier ce que votre projet n'est pas capable de réaliser, les services qui ne sont pas concernés par le projet, les fonctionnalités laissées à côtés. Ces points qui peuvent être intégré dans l'avenir.

5. Description fonctionnelle des besoins

Ce terme peu évocateur pour les non-initiés se résume simplement : décrivez votre besoin en termes de fonctionnalités . En quelques lignes, expliquez **ce que doit faire** votre application. C'est un point très important pour la réalisation. Comment pouvez-vous faire cette analyse ? Vous pouvez procéder de la sorte :

Prenez les fonctions principales et déclinez-les puis, pour chaque fonction, vous pouvez adopter identifier la description de la fonctionnalité, les contraintes (s'il y en a) / règles de gestion et le niveau de priorité. Si vous avez en tête une idée du design final, faites une maquette.

6. Budget pour la charge de l'entreprise

Précisez les ressources nécessaires à la réalisation dont le client ou l'entreprise cliente doit acheter pour que votre système marche bien. Ce budget est supporté par le client.

7. Budget de la main d'œuvre (coût du service/Produit logiciel)

Précisez les frais que vous estimez pour chaque étape du travail que vous développeur vous ferez. C'est une charge pour le client. Vous devez préciser aussi la modalité de paiement selon les différentes tranches.

8. Délais

Exprimez ici la date de réalisation attendue pour votre projet. Détaillez la chronologie (de chaque étape de réalisation du produit ou logiciel. Vous devez inclure toutes les étapes nécessaires des temps pendant lesquels vous aurez des obligations envers ce client (Ex : Formation des personnels, suivi et assistance du produit, ...)

9. Modalités de suivi et entretien

Ce sont les conditions dans lesquelles vous ferez le suivi post implantation du produit soit poste prestation. Rappelez la durée de suivi et entretien gratuit et le coût que le client supportera chaque fois après cette période.

10. Engagement entre vous développeurs et le client

Ecrivez un texte qui explique les situations possibles complexes qui peuvent arriver entre vous et donnez les pistes des solutions à aborder. Notez toutes les précisions et règles qui vont vous accompagner pendant la production du produit ou la prestation des services et même après.

11. Signatures

La date du jour, les noms et signatures des partenaires (vous développeurs et le client)

CHAP 2. ALGORITHMES

A. STRUCTURE DES DONNEES

Définition

Les structures de données sont un concept de programmation très important. Elles nous fournissent un moyen de stocker, d'organiser et de récupérer des données de manière efficace. Les structures de données sont utilisées pour faciliter le travail avec nos données.

Le fait d'utiliser une structure de données appropriée à un traitement informatique peut également faire baisser de manière significative la complexité d'une application informatique et ainsi contribuer à diminuer le taux d'erreurs.

Caractéristiques des structures de données

Les Data Structure sont souvent classées en fonction de leurs caractéristiques. Les caractéristiques possibles sont les suivantes :

- **Linéaire** ou **non linéaire** : Cette caractéristique décrit si les éléments de données sont disposés dans un ordre chronologique, par exemple avec un tableau, ou dans un ordre non ordonné, par exemple avec un graphe (Voir Graph Database).
- **Homogène** ou **non-homogène** : Cette caractéristique indique si tous les éléments de données d'un référentiel sont du même type ou de types différents (Textes, images, vidéos...).
- **Statique** ou **dynamique** : Cette caractéristique décrit la manière dont les structures de données sont compilées. Les structures de données statiques ont des tailles, des structures et des emplacements de mémoire fixes au moment de la compilation. Les

structures de données dynamiques ont des tailles, des structures et des emplacements de mémoire qui peuvent diminuer ou augmenter selon l'utilisation.

Types des structures des données

Différentes structures de données existent pour des données différentes ou répondant à des contraintes algorithmiques différentes. Ainsi nous pouvons citer selon différentes catégories :

- **Structures finies :**
 - constantes,
 - variables,
 - enregistrements,
 - structures composées finies ;
- **Structures indexées :**
 - tableaux (sur [1..n]),
 - tableaux multidimensionnels (e.g. tableaux bidimensionnels: sur [1..n, 1..m]; sinon, tableaux de tableaux: sur [1..n][1..m]).
- **Structures récursives :**
 - listes,
 - Piles
 - Files
 - Arbres,
 - Graphes.

Structure de donnée finies

1. Constantes : Linéaire, Homogène et statique

En informatique, une constante est un identificateur associé à une valeur fixe. Syntactiquement, cet identificateur a tous les aspects d'une variable. Cependant, il n'est possible de lui affecter une valeur qu'une seule fois, généralement au moment du lancement du programme.

❖ Exemple d'application en C# :

❖ **Limites exigeant d'utiliser plutôt la suivant :**

2. Variables : Linéaire, Homogène et statique

En informatique, les **variables** sont des symboles qui associent un nom (l'identifiant) à une valeur. Dans la plupart des langages et notamment les plus courants, les variables peuvent changer de valeur au cours du temps (dynamique). Dans les langages de certains paradigmes, notamment la programmation fonctionnelle, leur valeur est au contraire figée dans le temps (statique).

❖ **Exemple d'application en C# :**

❖ **Limites exigeant d'utiliser plutôt la suivant :**

3. Enregistrements

Un enregistrement (de l'anglais record) est un élément d'un tableau à deux dimensions ou d'une base de données. L'enregistrement contient habituellement plusieurs informations (entrées) qui se rapportent au même objet. Par exemple, un enregistrement d'un fichier contenant la description des clients d'une entreprise contiendra plusieurs informations (les rubriques) sur un client : son numéro de client, son nom, son adresse postale, son numéro de téléphone, etc.

❖ **Exemple d'application en C# :**

❖ **Limites exigeant d'utiliser plutôt la suivant :**

Structures indexées

1. Tableaux : Linéaire, Homogène et statique

Un tableau est une structure de données qui peut contenir plusieurs données de même type. Un élément du tableau est accédé en utilisant la variable du tableau suivie du numéro de l'élément appelé **indice (Index)**.

Le parcours de tout débutant en programmation commence par la résolution des questions relatives aux tableaux. Cette structure de données est utilisée dans toutes les situations possibles où on doit rassembler les objets en un seul endroit. Qu'il s'agisse d'un logiciel simple ou complexe ou d'une application Web, le tableau est principalement utilisé pour stocker et afficher les données de manière dynamique sur les pages Web.

Tableau à une dimension

Valeur	45	154	58	78	31	5	74
Index	0	1	2	3	4	5	6

Un tableau à une dimension, composé de 7 éléments.

Avec un tableau à une dimension (aussi appelé un vecteur), un numéro d'index donne accès à un seul élément. L'image de droite géométrique est une représentation graphique d'une telle structure de données, composée de 7 éléments.

En algorithmique, un tableau se déclare comme suit :

```
nomdutableau [valdebut..valfin] : Tableau de type
```

En langage C par exemple, on accède au premier élément d'un tableau nommé tab de la manière suivante :

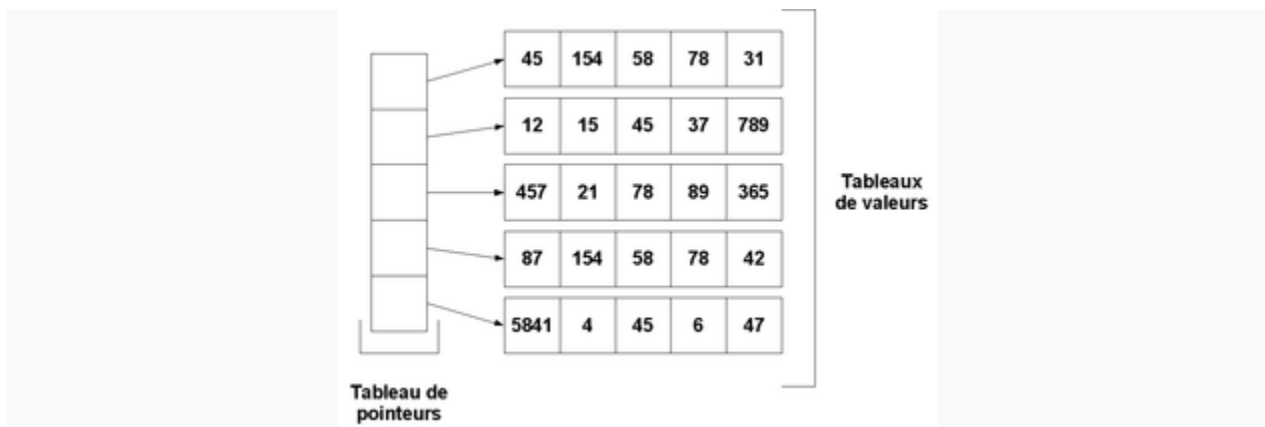
```
tab[0];
```


En prenant le tableau en exemple, cette instruction retournera 45. Il est important de savoir que la numérotation de l'index commence à 0.

❖ **Exemple d'application en C# :**

❖ **Limites exigeant d'utiliser plutôt la suivant :**

Tableau à 2 ou plusieurs dimension



Un tableau à deux dimensions, composé de 25 éléments

Un tableau à deux dimensions (aussi appelé matrice) est en fait un tableau normal (à une dimension) dont les éléments sont eux-mêmes des tableaux contenant les éléments du tableau à deux dimensions. On voit ceci sur l'illustration ci-contre, où chaque élément du premier tableau (vertical à gauche) est un tableau (horizontal).

L'accès aux éléments se fait à travers deux indices, d'abord un indice pour le premier tableau, ce qui nous en renvoie un autre tableau, auquel est affecté le deuxième indice. En prenant la représentation ci-contre, l'accès à la valeur située en 2e ligne et 5e colonne (la valeur 789), se fait via les indices 1 puis 4 (rappel : on commence la numérotation des indices à zéro).

Ce mécanisme d'imbrication peut être continué pour créer des tableaux à plus de deux dimensions. On peut créer un tableau à n dimensions, l'accès aux éléments nécessite alors une série de n indices.

L'ordre des indices est crucial. Dans l'exemple précédent, l'élément désigné par `tab[3][2]` (de valeur 58) diffère de l'élément indexé par `tab[2][3]` (de valeur 89).

❖ **Exemple d'application en C# :**

❖ **Limites exigeant d'utiliser plutôt la suivant :**

Structures récursives

1. Liste liée : Linéaire, Non-Homogène et dynamique

Une autre structure de données populaire et commune parmi les programmeurs est la liste chaînée. Les listes liées stockent des collections d'éléments dans un ordre linéaire. Chaque élément d'une liste liée contient un élément de données et un lien, ou une référence, vers l'élément suivant de la même liste.

Par exemple, lorsqu'on crée une liste de lecture sur un smartphone, elle fonctionne sur le concept de la liste liée. Ces chansons sont jouées une par une, elles sont connectées et on peut passer de la chanson trois à la chanson quatre mais pas revenir en arrière. C'est le comportement d'une liste singulièrement liée.

Lorsqu'on met en œuvre la fonctionnalité permettant de lire la chanson dans les deux sens, elle suit le comportement d'une liste doublement liée. Dans une liste doublement liée, les nœuds sont connectés dans les deux sens. La navigation bidirectionnelle est donc possible.

Lorsqu'on lit les chansons en mode répétition, elles suivent le comportement d'une liste circulaire liée. Dans une liste circulaire, le dernier nœud est connecté au premier nœud. Ainsi, une fois la dernière chanson terminée, la première chanson sera rejouée et la lecture se fera en mode cyclique, sans jamais s'arrêter.

❖ **Exemple d'application en C# :**

2. Pile (Stack) : Linéaire, Non-Homogène et dynamique

La pile suit une technique « LIFO » pour stocker et récupérer les éléments. L'élément qui est stocké à la fin sera le premier à être récupéré de la pile. La pile a les fonctions primaires suivantes :

Push() : Pour insérer un élément dans la pile.

Pop() : Pour retirer un élément de la pile.

Un éditeur de texte tel que Notepad ou Microsoft Word utilise une structure de données en pile pour accomplir des tâches d'annulation dans son éditeur. Un autre bon exemple de pile est le navigateur de votre ordinateur portable ou de votre système.

❖ **Exemple d'application en C# :**

3. Files : Linéaire, Non-Homogène et dynamique

Une file stocke une collection d'articles similaire à une pile ; cependant, l'ordre des opérations ne peut être que premier entré, premier sorti , FIFO (First In First Out). Lorsque l'on gère les accès à une ressource limitée à plusieurs personnes, on gère les priorités d'accès à cette

dernière. Par exemple, une imprimante avec mémoire reçoit un ensemble de documents à imprimer et doit les imprimer dans l'ordre d'arrivée, ou encore un système d'exploitation doit accorder à chaque programme du temps machine sans privilégier aucun, etc. Pour implémenter ce genre de comportement, on utilise des files de priorité qui se comportent comme des files d'attente et qui ont la propriété du type FIFO.

❖ **Exemple d'application en C# :**

4. Graphe : Non-Linéaire, Non-Homogène et statique

La structure de données graphe est une structure de données très puissante. Le graphe peut représenter non seulement la terre, mais aussi l'univers entier.

Lorsqu'on utilise Google Map, toutes les villes sont reliées comme un graphique avec des informations sur la distance. Il existe de nombreuses façons de se rendre d'une ville à l'autre, mais pour trouver le chemin le plus court entre deux villes, on doit utiliser certains algorithmes.

Pour décider du chemin le plus court vers votre destination, l'algorithme de Dijkstra active notre système de navigation/GPS dans notre téléphone. Uber utilise l'algorithme de Hungarian pour affecter chaque voiture aux personnes qui recherchent un trajet.

❖ **Exemple d'application en C# :**

5. Dictionnaire

6. Trie

À ne pas confondre avec un arbre, les tries sont également appelées arbres de mots-clés ou arbres de préfixes. Chaque fois qu'on utilise un moteur de recherche et que vous recevez des suggestions automatiques, ce sont des structures de données trie qu'on utilise.

❖ **Exemple d'application en C# :**

7. Table de hachage

Une table de hachage, ou une carte de hachage, stocke une collection d'éléments dans un tableau associatif qui trace les clés des valeurs. Une table de hachage utilise une fonction de hachage pour convertir un index en un tableau de cases qui contient l'élément de données désiré.

Une table de hachage est une structure de données permettant d'implémenter efficacement des dictionnaires. Bien que la recherche d'un élément dans une table de hachage puisse être aussi longue que la recherche d'un élément dans une liste chaînée ($O(n)$ dans le cas le plus défavorable), en pratique le hachage est très efficace.

8. Base de données

Une BD (Base de données) est un ensemble de données structurées modélisant un univers précis et accessible à plusieurs utilisateurs en même temps (Dennouni, 2017). Selon Bastien (2019) une base de données est une collection d'informations organisées afin d'être facilement consultables, gérables et mises à jour. Nous pouvons définir la BD comme étant une structure de données qui sert à conserver les informations d'une manière permanente afin de permettre la manipulation de celles-ci ultérieurement. En informatique, depuis leur apparition, les bases de

données ont connu 4 modèles majeures : BD hiérarchique, BD réseau, BD relationnelle, BD orientée objet (Lesleudis, 2018).

B. STRUCTURES DE TRAITEMENT

1. Les structures simples

a) Expressions de déclaration

C'est la première étape à faire lorsqu'on veut traiter les structures de données. Elle consiste à allouer une mémoire au sein du processeur (CPU) et la réserver au programme.

b) Expressions d'affectation ou de lecture (Read)

Cette action permet de ranger une nouvelle valeur dans une variable

Syntaxe

Expression $\leftarrow$ Valeur

Expression peut être :

- Une variable
- Une constante
- Une expression arithmétique
- Une expression logique

Remarque

- Une constante ne peut jamais figurer à gauche d'une affectation.
- Après une affectation, l'ancien contenu est perdu (scratchée) pour être substitué par le nouveau contenu.

- Une action d'affectation doit se faire entre deux types compatibles.

c) Expressions arithmétiques

Elles constituent toutes les opérations mathématiques qu'on effectue sur les structures de données :

Syntaxe : <exp-arith> opération_arith <exp-arith>

- opération_arith peut être '+', '-', '/', ou '*' Exemple : $(Y/2) + x*3$
- L'ordre de priorité des opérateurs arithmétiques :
 - { } crochets
 - [] accolades
 - () parenthèses
 - ^ puissance
 - * et / : multiplication et division
 - + et - : addition et soustraction

d) Expressions logiques

Les expressions logiques admettent Vrai ou Faux comme résultat. Elles peuvent utiliser des opérateurs relationnels (= , ?, <, <=, >, >=) ou des opérateurs logiques (NON, ET, OU) -

L'ordre de priorité est :

	>
	>=
	<
NON	<=
ET	=
OU	?

Exemple : $(x < 6)$ ET $(Y = 20)$ donne vrai si $x < 6$ et $Y = 20$ et faux sinon

e) Expression d'affichage ou d'écriture (write)

2. Les structures conditionnelles

En programmation, on est souvent confronté à des situations où on a besoin de choisir entre 2 ou plusieurs traitements selon la réalisation ou non d'une certaine condition d'où la notion de traitement conditionnel. On distingue deux structures de traitement conditionnel à savoir :

- La structure conditionnelle simple qui consiste à évaluer une condition (expression logique à valeur vrai ou faux) et d'effectuer le traitement relatif à la valeur de vérité trouvée.
- La structure conditionnelle à choix multiple qui consiste à évaluer une expression qui n'est pas nécessairement à valeur booléenne (elle peut avoir plus de deux valeurs) et selon la valeur trouvée, effectuer un traitement.

a) Les structures conditionnelles simples

1. Forme simple : (Si Alors Finsi)

Syntaxe :

Si condition **Alors** action(s)

Traitement

Fin si

Dans cette forme, la condition est évaluée. Si elle vaut vrai alors c'est la séquence d'actions qui est exécutée sinon c'est l'action qui suit l'action conditionnelle dans l'algorithme qui est exécutée.

Exemple 1 : Ecrire un algorithme qui permet de saisir un entier et d'afficher impossible d'être diviseur si cet entier est égal à 0.

2. Forme composée : (Si ... alors.....sinon)

Syntaxe :

Si condition **Alors**

Action(s)1

Sinon

Action(s)2

Fin si

Dans cette forme, la condition est évaluée. Si elle vaut vrai alors c'est la séquence d'actions 1 qui sera exécutée sinon c'est la séquence d'actions 2 qui sera exécutée.

Exemple 2 : Ecrire un algorithme qui permet de saisir un entier et d'afficher « pair » si cet entier est pair ou « impair » si cet entier est impair.

3.Forme imbriquée

Syntaxe :

- **Si condition 1 Alors**

Action(s)1

Sinon

Si condition 2 Alors

Action(s)2

Sinon

- **Si condition N-1 Alors**

Action(s)N-1

Sinon

Action(s)N

Fin si

Si la condition est vraie, alors la séquence d'actions 1 sera exécutée sinon on évalue la condition 2 si elle est vraie la séquence d'actions 2 sera exécutée. Enfin, si aucune des N-1 conditions est vraie alors on exécute la séquence d'actions N.

Exemple :

Ecrire un algorithme qui permet de saisir deux entiers A et B puis teste si $A > B$ ou $A < B$ ou $A = B$.

b) Les structures conditionnelles à choix multiples

Syntaxe

Selon <sélecteur> faire

<liste de valeurs1> : <traitement 1>

<liste de valeurs2> : <traitement 2>

.....

<liste de valeurs N> : <traitement N>

Sinon

<traitement N+1>

Fin selon

NB :

- Le sélecteur est un identificateur
- < traitement i > est une séquence d'actions.
- < liste de valeurs i > peut être une constante ou un intervalle de constantes de même type que sélecteur.

- La partie sinon est facultative. Elle est exécutée si aucune des valeurs n'est égale au sélecteur.

Exemple :

Ecrire un algorithme qui permet de lire un numéro de jour de la semaine (compris entre 1 et 7) et d'afficher le nom du jour en toute lettre.

3. Les structures itératives ou répétitives

On peut exécuter une action ou un ensemble d'actions non pas infiniment mais un certain nombre de fois : c'est la notion de boucles. Les éléments importants dans cette structure sont :

- Le compteur ;
- Le début de la boucle (Valeur initiale) ;
- La condition d'arrêt de la boucle (Valeur finale) ;
- Le pas d'avancement de la boucle et
- Le traitement à répéter.

Certains de ces éléments peuvent ne pas intervenir selon la structure répétitive utilisée.

Il existe les structures suivantes :

a) La structure « Pour faire.....Fin pour » : For....

Syntaxe :

Pour vc de vi à vf, pas de..., faire

Traitement

Fin pour

- Vc : compteur de type entier
- Vi et vf : valeur initiale et valeur finale de vc

- Traitement : action ou séquence d'actions à répéter ($v_f - v_i + 1$) fois.
- La boucle Pour est utilisée lorsque le nombre d'itération est connu à l'avance.
- V_c reçoit une valeur initiale v_i pour la première fois, il ne doit pas être modifié par une action de traitement à l'intérieur de la boucle.
- V_c est incrémenté automatiquement par le **pas** à chaque exécution du corps de la boucle Pour. Cette valeur d'incrément est appelée **le pas de la boucle**.
- L'exécution de la boucle finit lorsque le compteur vc atteint la valeur finale v_f .

Exemple :

Pour i de 1 à 5 (pas = 1), faire

Ecrire ($i * 100$)

Fin pour

⇒ Exécution : $i = 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6$

⇒ Résultat 100 200 300 400 500

Remarques :

- Une boucle peut être exécutée une ou plusieurs fois.
- Si le pas est différent de 1, il faut ajouter l'option (pas = constante)

Exemple

Pour i de 5 à 1 (pas = -1) faire

Ecrire ($i * 100$)

Fin pour

⇒ Exécution : $i = 5 \quad 4 \quad 3 \quad 2 \quad 1 \quad 0$

⇒ Résultat 500 400 300 200 100

Exemple 1

Ecrire un algorithme qui permet de calculer et d'afficher la somme des nb premiers entiers naturels (nb est saisi à partir de clavier).

Exemple 2

Ecrire un algorithme qui lit un entier n qu'on suppose positif puis affiche tous ses diviseurs.

b) La structure « Répéter Jusqu'à ce que..... » : Do....Until....

Syntaxe

Répéter

Traitement

Pas de la boucle

Jusqu'à (condition)

- Condition : condition d'arrêt et de sortie de la boucle
- Traitement : action ou ensemble d'actions à exécuter tant que la condition n'est pas vérifiée, dès qu'elle soit vérifiée, l'exécution du traitement s'arrête.
- Le nombre de répétition n'est pas connu à l'avance.
- Le traitement est exécuté au moins une fois quel que soit le résultat de la condition.
- La condition doit être initialisée avant le début de la boucle et doit être modifiée à l'intérieur de la boucle.
- Le pas de la boucle est défini à la fin du traitement
-

Exemple

i=1

Répéter

Ecrire ($i * 100$) $i ? i + 1$

Jusqu'à ($i > 5$)

⇒ Exécution : $i = 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6$

⇒ Résultat : 100 200 300 400 500

Exemple

Ecrire un algorithme qui permet de calculer la somme des nb premiers entiers en utilisant la boucle répéter jusqu'à

Exemple

Ecrire un algorithme qui permet de calculer la factorielle d'un entier n donné (on suppose que n est un entier positif)

c) La structure « Tant que..... Faire..... Fin tant que » : While (.....)

Syntaxe

Tant que (condition) faire

Traitement

Fin tant que

- Condition : condition de maintien de la boucle.
- Traitement : Action ou ensemble d'actions à exécuter tant que la condition est vérifiée.
- Le traitement est exécuté tant que la condition est vérifiée sinon on sort de la boucle.
- Si la condition n'est pas vraie dès la première exécution, la boucle ne sera jamais exécutée (0 fois).
- Le nombre de répétition n'est pas connu à l'avance.

- La condition doit être initialisée avant la boucle et modifiée à l'intérieur de la boucle.

Exemple

i = 1

Tant que (i>5)

Ecrire (i *100) i? i +1

Fin tant que

⇒ Exécution : i = 1 2 3 4 5 6

⇒ Résultat 100 200 300 400 500

Exemple

Reprendre les exercices précédents en utilisant la boucle tant que.

C. CONCEPTION D'ALGORITHMES

Avez-vous déjà eu l'occasion de faire la cuisine ? Pour un gâteau, vous êtes-vous lancé directement ou avez-vous ouvert un livre pour récupérer la liste et la quantité de chaque ingrédient, pour suivre la recette : faites fondre le chocolat et le beurre dans une casserole à feu doux, retirez la casserole du feu, incorporez les jaunes d'œuf, puis le sucre et la farine, battez les œufs en neige puis incorporez doucement dans le mélange, etc.

Dans ce cas, félicitations ! Vous avez déroulé votre premier algorithme !

Définition

En informatique, un algorithme est une suite logique d'opérations ou d'instructions aboutissant à la résolution d'un problème. Un algorithme décrit une suite finie d'opérations à

appliquer dans un ordre déterminé pour résoudre un problème. Un algorithme peut être traduit, grâce à un langage de programmation, en un programme exécutable par un système informatique (ordinateur, carte microprocesseur, objet connecté).

L’algorithmique sert à réfléchir à l’algorithme :

- Trouver les bonnes variables
- Imbriquer les boucles dans le bon sens
- Enchaîner les si-alors sinon dans le bon ordre
- Etc.

L’algorithme d’une voiture sans conducteur va devoir piloter :

- la mise en marche, la direction et l’arrêt
- la détection des lignes délimitant la route
- la détection et l’évitement des obstacles

Cet algorithme est traduit, grâce à un langage de programmation, en un programme exécutable par un système informatique (ordinateur, carte microprocesseur, objet connecté...).



Etapes d’élaboration

Quelles sont les étapes de l’élaboration de l’algorithme et du programme ?

Etape 1 : Identifier les entrées de l'algorithme, ses sorties ;

Etape 2 : Ecrire l'algorithme en langage naturel : suite logique d'opérations ou d'instructions, souvent rédigées sur feuille de papier en utilisant des mots clés : si, alors, tant que, jusqu'à...

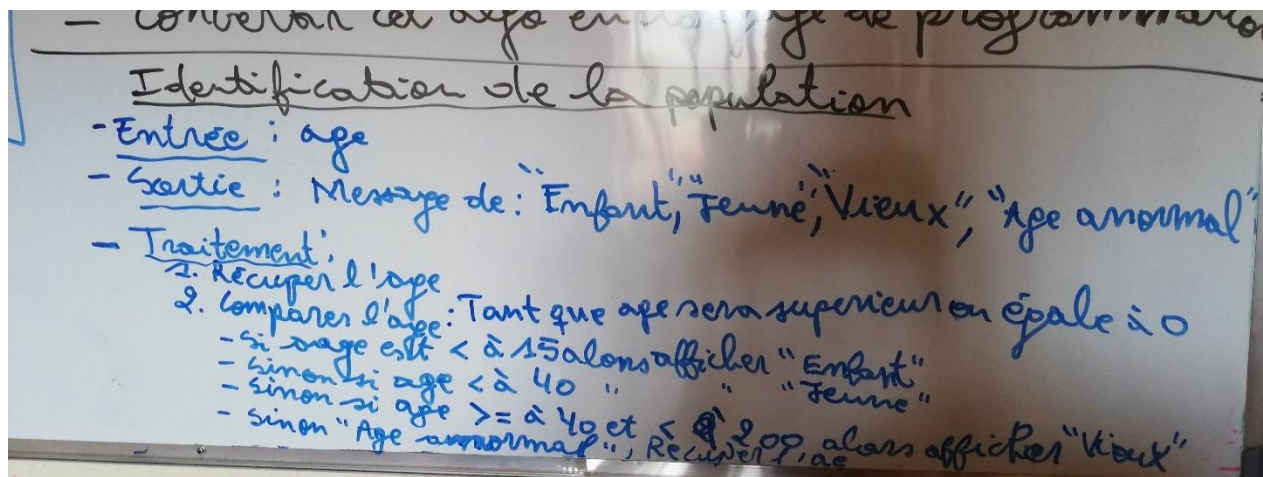
Etape 2 : Construire une représentation graphique de l'algorithme (c'est mieux de le faire l'aide d'un logiciel).

Etape 3 : A partir de la représentation graphique, le logiciel traduit l'algorithme en langage de programmation pour que l'objet puisse exécuter le programme.

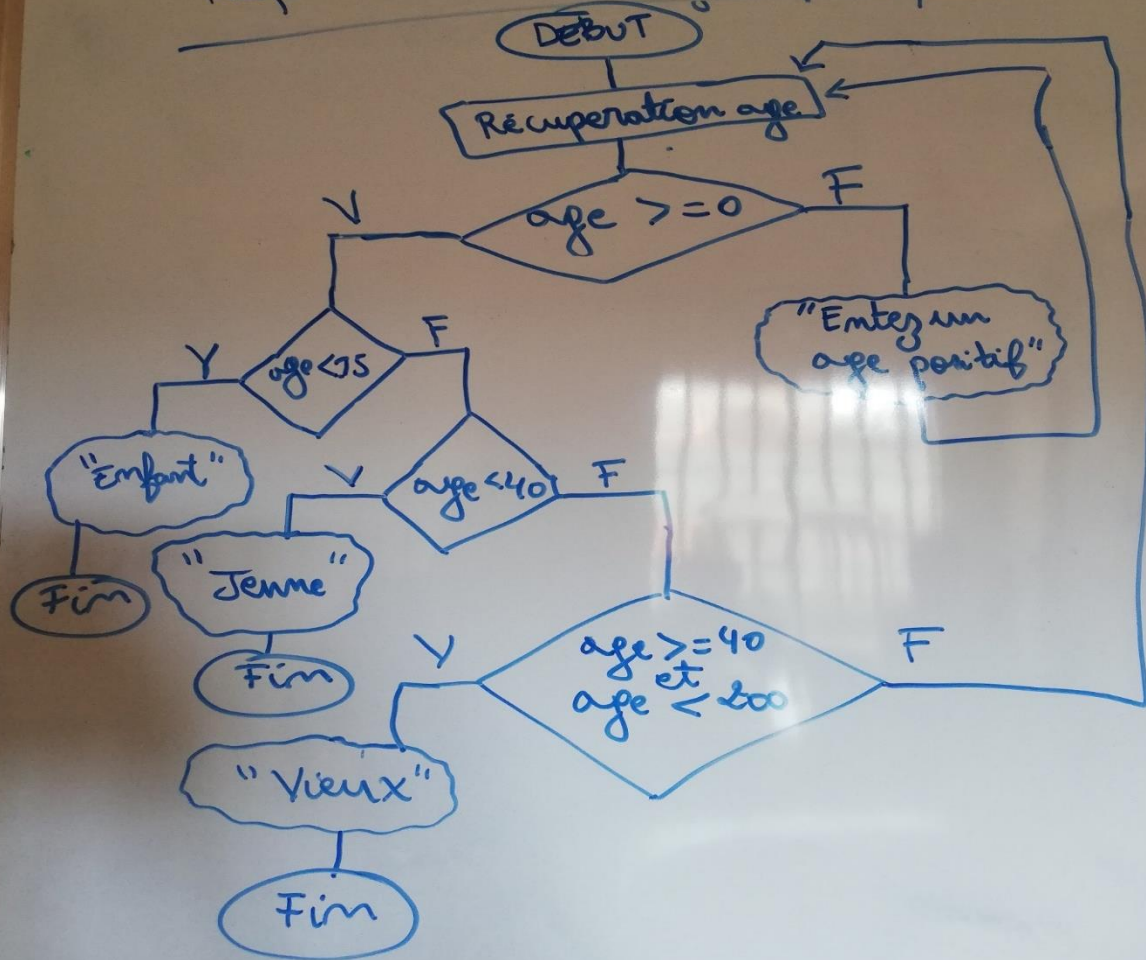
Exemple d'application :

Faire un algorithme pour un programme qui récupère l'âge de l'individu puis dit si cet individu est Enfant, soit Jeune soit Vieux.

Résolution :



Représentation graphique



CHAP 3. CODAGE DES ALGORITHMES ET PROGRAMMATION DANS UN LANGAGE DONNE

A. Les sous programmes

Un sous-programme est une portion de code analogue à un programme. Déclarée dans un programme ou dans un sous-programme et dont la partie instruction peut être exécutée plusieurs fois au cours du traitement du programme grâce à des appels.

Voici les buts de l'utilisation des sous programmes :

- décomposition des problèmes en modules (sous problèmes de taille réduite) : Dans un programme plusieurs séquences d'instructions sont appelées plusieurs fois et depuis divers points du programme. Il serait donc plus intéressant d'isoler ces séquences dans un sous-programme qui peut être appelé depuis n'importe quel point du programme.
- L'approche modulaire réduit énormément le nombre d'instructions redondantes (qui se répètent) moyennant l'ajout d'une séquence d'appel pour le module à différents endroits du programme. D'où la réduction de la taille du programme (code).
- La lisibilité qui facilite notablement la compréhension du programme
- Réutilisation du sous-programme

En résumé, le programme sera plus lisible et plus facile à maintenir (à modifier éventuellement par la suite). On distingue deux formes de sous programmes : **Les procédures** et **les fonctions**.

1. Les procédures

Une procédure est un sous-programme qui effectue un traitement (suite d'instructions) sans être obligé de retourner (ou donner en retour) une valeur. On peut utiliser une procédure sans paramètre soit avec paramètre.

a) Les procédures sans paramètre

Elles sont utilisées pour éviter d'avoir à réécrire plusieurs fois une même suite d'instructions figurant plusieurs fois dans le programme.

Déclaration

Procédure nom_procédure

Début

Suite d'instructions

Fin

Appel (utilisation)

nom_procédure ()

b) Les procédures avec paramètres

Ce sont des procédures qui demandent des valeurs (paramètres) pour s'exécuter. Ces paramètres interviennent dans le traitement du sous-programme. Quand on appelle la procédure, les paramètres s'appellent des arguments (**paramètres affectifs**) alors qu'ils sont appelés **paramètres formels** lors de la déclaration de la procédure. Les paramètres affectifs doivent être des valeurs correspondantes et compatibles aux paramètres formels.

Une déclaration de procédure peut comporter toute une liste de paramètres dont la syntaxe est la suivante.

Procédure nom_procédure (liste de paramètres formels).

Exemple

Déclaration

Procédure Somme_trois_valeur (a, b, c : entier)

Début

c<-a+b

Fin

⇒ Là les paramètres sont appelés **formels**

Appel (utilisation)

Somme_trois_valeur (valeur de a, valeur de b, valeur de c)

⇒ Là les paramètres sont appelés **affectifs**.

2. Les fonctions

Une fonction est un sous-programme qui renvoie une valeur d'un seul type, c'est une procédure qui retourne une valeur d'un type selon que la fonction a été déclarée. Ce type sera celui de la fonction. Elle a les mêmes propriétés et principes (paramètres) que la procédure.

- Fonction sans paramètres :

Déclaration

Fonction nom_Fonction : type de la fonction (→ type de valeur de retour)

Début

Suite d'instructions

Retourner valeur (➔ de même type que la fonction)

Fin

Appel (utilisation)

Variable = nom_Fonction ()

- Fonction avec paramètres

Déclaration

Ex : Procédure Somme_trois_valeur (a, b, c : entier) : type de la fonction

Début

d:entier = a+b*c

Retourner d (de meme type que la fonction)

Fin

⇒ Là les paramètres sont appelés **formels**

Appel (utilisation)

Afficher Somme_trois_valeur (valeur de a, valeur de b, valeur de c)

⇒ Là les paramètres sont appelés **affectifs**.

B. La récursion

Une des caractéristiques les plus importantes de programmation est la possibilité pour une **procédure** ou une **fonction** de s'appeler elle-même. On parle de **récursivité**.

La récursivité est particulièrement utile pour traiter tous les problèmes formalisables de façon récursive, bien qu'il soit possible de programmer des solutions n'utilisant pas la récursivité pour ces problèmes.

Syntaxe

Fonction Récursive (par : type_par) : typer_retour

Variable résultat : type_retour

Début

 Si (cdt_arret) alors

 résultat ← val_initiale

 sinon Appel de la fonction Récursive (par2)

fin si

 retour (résultat)

Fin

C. Méthodes de programmation

La mise sur pied d'un programme a toujours exigé le respect d'énormes règles informatiques, appelée aussi méthodes de programmation informatiques, qui ne sont d'autres que des méthodes utilisées par un informaticien et d'autre part le langage lié à cette méthode.

Parmi les méthodes de programmation, nous distinguons :

- La programmation classique ;
- La programmation structurée et modulaire ;
- La programmation orientée objet.

1. Programmation classique

C'est la programmation où les instructions sont suivies les unes après les autres.

2. Programmation structurée et modulaire

C'est la programmation où on fait intervenir les sous programmes : procédures et fonctions, c'est une méthode de décomposition du code en tâches et sous- tâches. Ceci le rend plus décomposable. Le programme est plus clair, plus lisible, plus aisément modifiable...

Voici comment il faut procéder :

- Identifier précisément le problème que le programme se doit de résoudre
- Décomposer le problème en sous-problèmes
- Définir les fonctions correspondantes
- Écrire le programme principal en pseudo-code
- Implémenter, exécuter le programme avec différentes données,

3. Programmation orientée objet

La programmation orientée objet (POO) est **un paradigme au sein de la programmation informatique**. Il s'agit d'une représentation des choses, un modèle cohérent – partagé à travers différents langages qui permettent son usage (Python, Java, C++, C#, ...). Le but de la POO consiste à définir et faire interagir entre eux des objets, compris ici comme tous types de structures issues d'un langage donné. Toutefois, par convenance, les objets désignent le plus souvent **des variables complexes**, elles-mêmes composées de variables ou de fonctions.

C'est une méthode consistant à subdiviser les codes en classes d'objets. Dans cette méthode Une application est vue comme un ensemble d'objets qui interagissent. Comme les objets du monde (une voiture, un vélo, une table, ...), les objets dans les langages OO ont des états et des comportements. Une voiture est caractérisée par sa couleur, sa vitesse instantanée, sa consommation : ce sont **ses états** et par des actions telles que tourner, accélérer, s'arrêter : ce sont **ses comportements**.

❖ Les classes

Plusieurs objets peuvent être de la même famille (type)
On dit qu'ils appartiennent à la même classe. Une classe définit les variables et les méthodes communes à tous les objets de la classe. Un objet appartenant à une classe est dit instance de cette classe. Ainsi, après avoir créé une classe, il faut instancier (créer une instance) avant de pouvoir l'utiliser.

Exemples

Class : BankAccount

Object : acount_001_210

Class : GSM(Type, Num Identification)

Object : mon_GSM (Nokia, 1235321)

❖ Les objets

Un objet logiciel regroupe des variables et des méthodes (fonctions). L'état d'un objet est défini par les valeurs de ses variables. C'est une instance de la classe. Les comportements d'un

objet sont ses méthodes. Ces variables et méthodes sont appelées variables et méthodes d'instance (par opposition aux variables et méthodes de classe).

Exemples d'objets

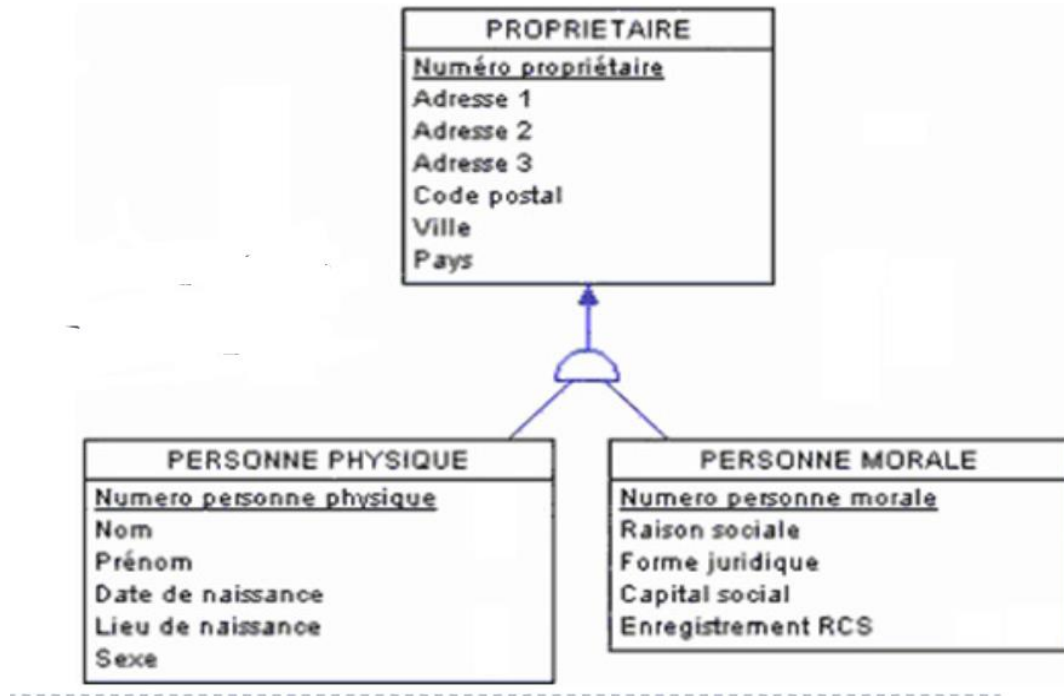
- Une personne est un objet caractérisé par un nom, un prénom, une date de naissance, une profession ... et pouvant marcher, parler, réfléchir, écrire, ...
- Un compte en banque est un objet caractérisé par un numéro de compte, un propriétaire, un taux d'intérêt, un solde, un solde minimum, ... et offre des services : consultation du solde, retrait, virement ...
- Un objet graphique est un objet caractérisé par une taille, par une position, une forme, une couleur ... et peut changer de taille, de forme, de couleur ...

❖ Principe d'héritage

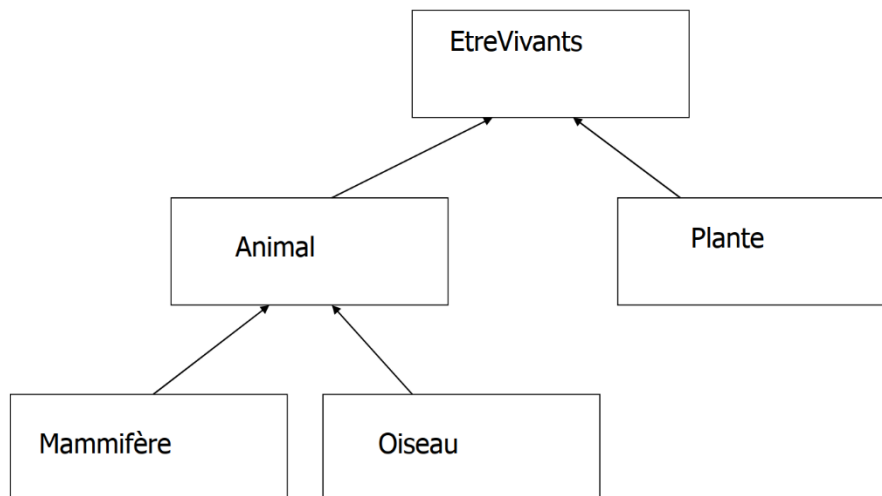
L'héritage est le mécanisme qui permet de créer une classe à partir d'une classe existante. La nouvelle classe hérite les variables et les méthodes de sa classe parente. Elle peut avoir d'autres variables et méthodes qui lui sont propres. Une classe peut hériter d'un ou de plusieurs classes. Ceci aide à factoriser le code.

Exemple

PERSONNE PHYSIQUE et PERSONNE MORALE sont deux classes qui héritent de la classe PROPRIETAIRE



Exemple d'hérarchie d'héritage



❖ L'Encapsulation des classes

L'encapsulation est un concept important en programmation orientée objet. L'encapsulation est un mécanisme consistant à rassembler les données et les méthodes au sein d'une structure en

cachant l'implémentation de l'objet, c'est-à-dire en empêchant l'accès aux données par un autre moyen que les services proposés. L'encapsulation permet donc de garantir l'intégrité des données contenues dans l'objet. Ainsi, si l'on veut protéger des informations contre une modification inattendue, on doit se référer au principe d'encapsulation.

L'encapsulation permet de définir des niveaux de visibilité des éléments de la classe. Ces niveaux de visibilité définissent les droits d'accès aux données selon que l'on y accède par une méthode de la classe elle-même, d'une classe héritière, ou bien d'une classe quelconque. Il existe quatre niveaux de visibilité :

- Visibilité par défaut : aucun modificateur de visibilité n'est indiqué.
- Visibilité publique : les fonctions de toutes les classes peuvent accéder aux données ou aux méthodes d'une classe définie avec le niveau de visibilité « public ». Il s'agit du plus bas niveau de protection des données.
- Visibilité protégée : l'accès aux données est réservé aux fonctions des classes héritières, c'est-à-dire par les fonctions membres de la classe et des classes dérivées. Ainsi, Un attribut ou une méthode déclarée "protected" est accessible uniquement aux classes d'un package et à ses sous-classes même si elles sont définies dans un package différent.
- Visibilité privée : l'accès aux données est limité aux méthodes de la classe elle-même. Il s'agit du niveau de protection des données le plus élevé

POO : récapitulatif

- Tout est un objet
- L'exécution du programme est réalisée par échange de messages entre objets

- Tout objet est une instance de classe, qui est le "moule" générique des objets de ce type
- Les classes définissent les comportements possibles de leurs objets
- Les classes sont organisées en une structure arborescente à racine unique : la hiérarchie d'héritage
- L'encapsulation est la méthode d'accès et de protection des classe.

❖ Pratiques de la programmation en C#

CHAP 4. Tests et validation de l'application